강화학습 기반 유저 적응형 협동 전투 NPC 시스템

User-Adaptive Cooperative Combat NPC System Based on Reinforcement Learning

손승현
Seunghyun Son
서강대학교 가상융합전문대학원
Sogang Univ.
sonsh980318@gmail.com

박준형
Junhyeong Park
서강대학교 가상융합전문대학원
Sogang Univ.
pjh7083@sogang.ac.kr

최승관
Seunggwan Choe
서강대학교 가상융합전문대학원
Sogang Univ.
csk0123@sogang.ac.kr

요약문

본 연구에서는 플레이어의 숙련도에 따라 협동 방식을 조절하는 NPC를 강화학습으로 생성하는 파이프라인을 제안한다. 기존의 규칙 기반 AI(Behavior Tree, FSM 등)는 예측 가능한 행동 패턴으로 인해 몰입감이 저하되고, 난이도 조절을 위해 수많은 규칙을 수동으로 조정해야 하는 한계가 있다. 본 연구에서는 2단계 학습 파이프라인을 설계하였다. 1단계에서는 PPO 알고리즘과 파라미터 공유 기반 셀프플레이를 통해 균형 잡힌 전투 AI를 생성하며, 학습 과정의 체크포인트를 저장하여 숙련도별 플레이어 모델로 활용한다. 2단계에서는 1단계 체크포인트를 플레이어 역할로 배치하여 해당 숙련도에 맞춰 협동하는 NPC를 학습한다. 이를 통해 초보 플레이어에게는 적극적으로 전투를 주도하는 NPC를, 숙련된 플레이어에게는 보조 역할에 집중하는 NPC를 제공한다. 20x20 격자 기반 5vs5 전투 환경에서 12,000 에피소드 학습 후 양 팀 승률이 50:50으로 수렴하였으며, UDP 기반 미러링 아키텍처를 통해 학습된 PyTorch 모델을 ONNX 변환 없이 Unity에서 실시간으로 활용할 수 있다.

주제어

협동 NPC, 유저 적응형 AI, 강화학습, 난이도 조절, 멀티에이전트, 게임 AI

1 서론

게임 산업에서 NPC(Non-Player Character)의 지능적 인 행동은 플레이어 경험의 핵심 요소이다. 특히 팀 기반 전투 게임에서 유저와 함께 싸우는 협동형 NPC는 게임의 몰입감과 재미를 좌우하는 중요한 요소이다. 그러나 전통적인 규칙 기반 AI(Behavior Tree, FSM 등)는 정해 진 규칙에 따라 행동하여 반복 플레이 시 패턴이 노출되고, 난이도별로 수많은 규칙을 수동으로 조정해야 하며, 유저의 플레이 스타일에 맞춰 행동을 조정하지 못하는 한계가 있다.

강화학습 기반 NPC는 이러한 문제를 해결할 수 있는 잠재력을 가진다. 그러나 기존의 멀티에이전트 강화학습 연구들은 "인간을 이기는 강력한 AI" 생성에 집중하여, 게임 디자인 관점에서 중요한 난이도 조절과 유저와의 협동에 대한 연구가 부족하다. 게임 개발 현장에서 필요한 것은 플레이어를 압도하는 AI가 아니라, 적절한 도전감을 제공하면서 유저와 시너지를 발휘하는 "함께 싸우는 동료" NPC이다.

본 연구에서는 이러한 문제를 해결하기 위해 플레이 어의 숙련도에 따라 협동 방식을 조절하는 유저 적응형 NPC 학습 방법을 제안한다. 2 단계 학습 파이프라인을 통해, 1 단계에서 균형 잡힌 전투 AI 를 생성하며 학습 과정의 체크포인트를 숙련도별 플레이어 모델로 활용하고, 2 단계에서 해당 체크포인트를 플레이어 역할로 배치하여 협동 NPC 를 학습한다. 이를 통해 초보 플레이어에게는 전투를 주도하는 적극적인 NPC 를, 숙련된 플레이어에게는 보조 역할에 집중하는 NPC 를 제공할 수 있다. UDP 미러링 아키텍처로 학습된 모델을 Unity 게임에 즉시 적용할 수 있다

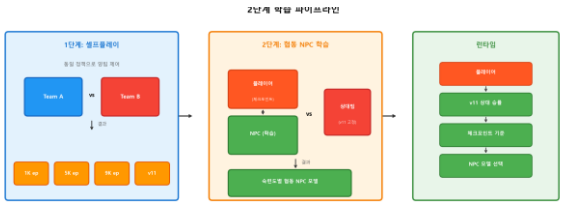

그림 1. 2 단계 학습 파이프라인 개요. 1 단계 셀프플레이로 균형 모델(v11)과 숙련도별 체크포인트를 생성하고, 2 단계에서 체크포인트를 플레이어 역할로 배치하여 협동 NPC 를 학습한

다. 플레이어 숙련도 판별은 v11 상대 승률을 기준으로 수행된다.

2 관련 연구

2.1 멀티에이전트 강화학습

OpenAI Five 는 Dota 2 에서 5vs5 멀티에이전트 환경에서 셀프플레이를 통해 프로 수준의 플레이를 달성하였다[1]. DeepMind 의 AlphaStar 는 StarCraft II 에서 그랜드마스터 수준을 달성하며 멀티에이전트 학습의 가능성을 보여주었다[2]. Lowe et al 은 협력-경쟁이 혼합된 환경에서 중앙집중식 학습과 분산 실행(CTDE)방식의 Multi-Agent Actor-Critic 알고리즘(MADDPG)을 제안하였다[4]. 또한 Baker et al 은 숨바꼭질 환경에서 셀프플레이를 통해 에이전트들이 도구 사용과 같은 창발적 전략을 학습할 수 있음을 보여주었다[5].

2.2 게임 AI 와 강화학습

PPO(Proximal Policy Optimization)는 안정적인 학습과 샘플 효율성으로 게임 AI 분야에서 널리 사용된다[3].

파라미터 공유(Parameter Sharing) 기법은 동일한 정책 네트워크를 여러 에이전트가 공유하여 학습 효율을 높이는 방법으로 활용된다.

최근 연구에는 RPG 게임의 전투 NPC 를 멀티에이전트 강화학습으로 학습시키는 시도가 이루어졌다[8]. 해당 연구는 DQN, PPO, OMIX 등 다양한 알고리즘을 비교하여 인간 플레이어를 이기는 강력한 AI 를 생성하였다. 그러나 이러한 연구들은 학습 알고리즘의 성능 비교에 집중하여, 학습된 모델을 실제 게임 엔진에 배포하는 방법이나 게임 디자인 관점에서의 난이도 조절에 대한 고려가 부족하다. 본 연구는 학습 성능뿐 아니라 Unity 게임 엔진으로의 배포 파이프라인과 난이도 조절 가능한 NPC 생성에 초점을 맞춘다.

2.3 Unity 에서의 강화학습 현황

Unity 는 ML-Agents Toolkit 이라는 오픈소스 프레임워크를 통해 게임 환경에서 강화학습 에이전트를 학습시킬 수 있도록 지원한다[6]. ML-Agents 는 PyTorch 기반의 PPO, SAC 등의 알고리즘을 제공하며, 학습된 모델은 Unity Sentis 추론 엔진을 통해 게임 내에서 실행할 수 있다.

그러나 ML-Agents 에는 시뮬레이션 속도가 최대 100 배속으로 제한되어 대규모 학습 시 병목이 되고, IL2CPP 스크립팅 백엔드에서 학습이 지원되지 않으며, Headless 모드에서 시각적 관찰을 수집할 수 없는 등의 기술적 제약이 존재한다.

표 1 은 Unity ML-Agents 와 본 연구에서 제안하는 순수 Python 기반 학습 방식의 비교이다.

표 1. 학습 환경 비교 : Unity ML-Agents vs 제안 방식

항목	ML-Agents	제안 방식
시뮬레이션 속도	최대 100배속	제한 없음
학습 처리량	환경 의존적	~905 FPS
렌더링 오버헤드	있음	없음
PyTorch 직접 사용	제한적	완전 지원
디버깅 환경	Unity/C#	Python

본 연구는 ML-Agents 의 제약을 우회하여 순수 Python 환경에서 빠르게 학습을 수행하고 UDP 통신으로 Unity 와 실시간 연동하는 방식을 제안한다.

3 연구 방법

3.1 환경 설계

본 연구에서는 20x20 격자 기반 2D 전투 환경을 구현하였다(그림 2). 각 팀은 5 개의 유닛으로 구성되며, 역할별로 차별화된 스탯과 고유 스킬을 가진다(표 2). 맵에는 벽(10%), 위험 타일(5%), 버프 타일(3%)이 배치되며, 벽은 좌우 대칭으로 생성하여 양 팀의 지형적 공정성을 보장한다.

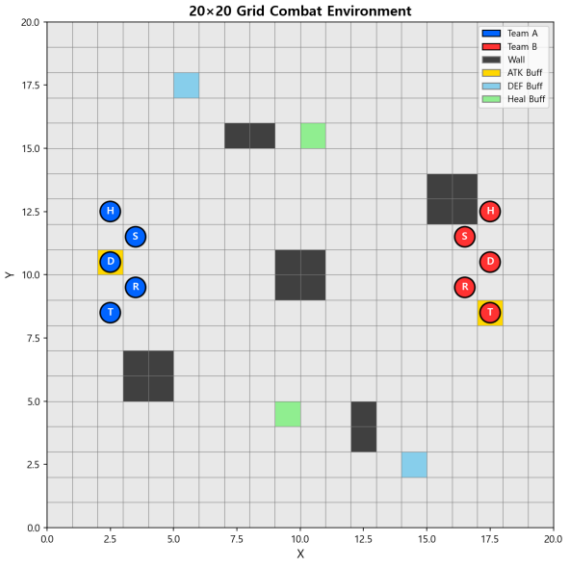

그림 2. 학습 환경 시각화. 20x20 격자 맵에서 Team A(파랑)와 Team B(빨강)가 전투를 수행한다. 회색은 벽, 초록/노랑/분홍은 버프 타일을 나타낸다.

표 2. 역할별 유닛 스탯 및 고유 스킬

역할	HP	MP	공격	방어	사거리	스킬
탱커	150	30	10	15	1	도발
딜러	80	50	25	5	1	범위공격
힐러	70	100	8	5	2	치유
레인저	60	60	20	3	4	관통샷
서포터	90	80	12	8	2	버프

3.2 관찰 공간 설계

각 에이전트는 기본 233 차원의 관찰 벡터를 입력받으며, 유저 협동 모드에서는 플레이어 정보 6 차원이 추가되어 총 229 차원으로 확장된다 (표 3). 셀프플레이에서 양팀이 동일한 정책 네트워크를 공유하므로, 관찰 벡터의 대칭성 보장이 학습 안정성에 중요하다.

표 3. 관찰 벡터 구성(협동 모드 229 차원)

구성 요소	차원	설명
자기 상태	10	HP, MP, 위치, 생존, 역할(5)
아군 상태	40	4명 × 10차원
적군 상태	50	5명 × 10차원
지형 정보	121	11×11 패치
전역 정보	2	현재 턴, 팀 ID
플레이어 정보	6	one-hot 역할 인코딩 (협동 모드)

플레이어 정보는 [없음, 탱커, 딜러, 힐러, 레인저, 서포터]의 6 차원 one-hot 벡터로 인코딩되어, NPC 가 플레이어의 역할을 인식하고 그에 맞는 협동 행동을 학습할 수 있도록 한다.

대칭성 보장을 위해 위치는 중앙 기준 정규화$X_{norm} = \frac{2x-(W-1)}{W-1}$를 적용하고, 팀 ID 는 Team A 를 −1, Team B 를 +1 로 원점 대칭 인코딩하며, 아군/적군 리스트를 거리순으로 정렬하여 고정 순서 편향을 제거하였다.

3.3 행동 공간 설계

행동 공간은 12 개의 이산 행동으로 구성된다 (표 4). 역할별 고유 스킬은 해당 역할의 유닛만 사용 가능하며, MP 부족이나 쿨타임 등으로 무효한 행동 선택 시 −0.5 의 패널티를 부여하며 정책이 유효한 행동을 학습하도록 유도한다.

표 4. 행동 공간 구성(12 개)

ID	행동	사용 조건
0	제자리 대기	모든 역할
1−4	이동 (상/하/좌/우)	모든 역할
5	가까운 적 공격	모든 역할
6	최저HP 적 공격	모든 역할
7	도발 (방어+10, 3턴)	탱커 전용
8	범위공격 (15dmg, 2칸)	딜러 전용
9	치유 (45HP, 3칸)	힐러 전용
10	관통샷 (18dmg, 2명)	레인저 전용
11	버프 (공격+5, 3칸)	서포터 전용

동시 행동 처리 : 양 팀의 공격이 동시에 처리되도록 구현하였다. 모든 공격 의도를 먼저 수집한 후 데미지를 일괄 적용하여, 특정 팀의 선공 이점을 제거하고 공정한 전투를 보장한다.

3.4 보상 설계

보상 함수는 희소 보상, 밀집 보상, 역할별 보상의 계층 구조로 설계하였다:

$$R_{total} = R_{sparse} + R_{dense} + R_{role} \quad (1)$$

희소 보상(R_{sparse})은 에피소드 종료 시 승리 +15(전멸 시 추가 +5), 패배 −15, 무승부 −5 를 제공한다.

3.4.1 밀집 보상 및 소극적 전투 방지

매 스텝 행동에 대한 즉각적 피드백과 함께, 강화학습에서 흔히 발생하는 소극적 회피 행동을 방지하기 위한 보상을 통합 설계하였다 (표 5). 특히 적과의 거리가 8 칸을 초과할 경우 거리에 비례한 패널티를 부여하여 회피 전략을 억제하였다.

표 5. 밀집 보상 및 소극적 전투 방지 보상

보상 유형	값	목적
적 처치	+5.0	공격 장려
데미지 (per HP)	+0.3	공격 행동 유도
힐 (per HP)	+0.1	치유 행동 유도
적 접근	+0.3	전투 참여 유도
전투 중	+0.15/턴	교전 지속 장려
버프 타일	+0.2	전략적 위치 활용
사망	−0.3	생존 유도
제자리 대기	−0.5	소극적 행동 억제
무효 행동	−0.5	유효 행동 학습
시간 경과	−0.05/턴	빠른 교전 유도
무승부	−8.0	장기전 회피
적 거리 8칸 초과	$-0.1 \times (d-8)$	회피 억제

3.4.2 역할별 보상

각 역할의 특성에 맞는 행동을 유도하기 위한 추가 보상이다 (표 6). 탱커(−2.0), 힐러(−1.5), 딜러(−1.0) 순으로 사망 패널티를 차별화하여 팀 구성의 중요성을 반영하였다.

표 6. 역할별 추가 보상

역할	보상 조건	값
탱커	피격/도발/사망	+0.05/HP, +0.3, −2.0
딜러	데미지/AOE/사망	+0.2/HP, +0.2, −1.0
힐러	힐/사망	+0.15/HP, −1.5
레인저	원거리(2칸+) 공격	+0.1/HP
서포터	아군 3칸 내 위치	+0.03/명

3.4.3 협동 보상

초기 설계에서 적 처치 보상을 가장 높게 설정하였을 때, NPC 가 플레이어를 무시하고 독단적으로 적에게 돌진하는 문제가 발생하였다. 이를 해결하기 위해 플레이어와의 거리 유지에 칸별 보상을 부여하고, 적이 사거리 내에 진입했을 때 전투 및 처치 시 보상을 제공하도록 재설계하였다 (표 7).

표 7. 협동 보상 설계

보상 유형	값	적용 조건
플레이어 근접 유지	+0.3/칸	플레이어 3칸 이내 (거리 반비례)
플레이어 이탈	−0.5/칸	플레이어 5칸 초과 시
사거리 내 전투	+1.0	적이 공격 범위 진입 시 공격
사거리 내 처치	+3.0	사거리 내 적 처치 시
플레이어 보호	+1.5	탱커가 플레이어 앞에 위치

3.5 정책 네트워크

PPO(Proximal Policy Optimization) 알고리즘을 기반으로 파라미터 공유 방식의 멀티에이전트 학습을 수행하였다 (그림 3). PPO 는 다음의 클리핑된 목적 함수를 최대화한다[3]:

$$L^{CLIP}(\theta) = \mathbb{E}_t[min(r_t A_t, \bar{r}_t A_t)] \quad (2)$$

여기서 $r_t = \pi_\theta(a_t|s_t)/\pi_{\theta_{old}}(a_t|s_t)$는 정책 비율, $\bar{r}t$ = clip(rt, $1-\epsilon$, $1+\epsilon$) 는 클리핑된 비율, $\hat{A}t$ 는 추정된 어드밴티지이다. 클리핑을 통해 정책 업데이터의 급격한 변화를 방지하여 안정적인 학습을 보장한다.

주요 하이퍼파라미터는 다음과 같다: 학습률 3×10−4, 감가율 γ = 0.99, GAE λ = 0.95, 클리핑 ϵ = 0.

4 실험

4.1 실험 환경

본 연구의 학습 실험은 표 8 의 하드웨어 환경에서 수행되었다.

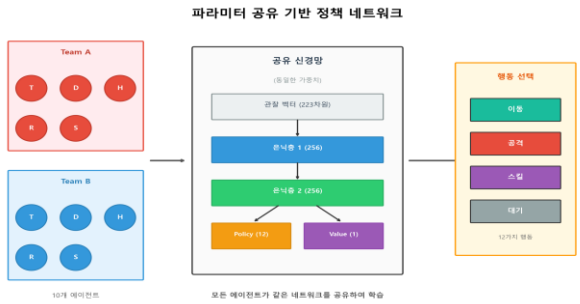

그림 3. 정책 네트워크 구조. 관찰 벡터(223 차원)를 입력받아 2 개의 은닉층(256 유닛)을 거쳐 행동 확률(12 차원)과 상태 가치를 출력한다.

표 8. 실험 환경

항목	사양
CPU	Intel Core i7-14700F
GPU	NVIDIA GeForce RTX 4060 Ti
RAM	32GB DDR5
OS	Windows 11
Python	3.10
PyTorch	2.0

4.2 1 단계 : 동등 조건 셀프플레이

양 팀이 동일한 스탯(100%)을 가진 조건에서 동일한 정책 네트워크가 양 팀을 제어하는 셀프플레이 학습을 진행하였다. 표 9 과 그림 4 에서 보듯이, 초기에는 Team B 가 90%까지 우세했으나 약 2,700 에피소드(4.8M steps)부터 균형을 이루기 시작하여 최종적으로 50.2:49.4 로 수렴하였다.

표 9. 1 단계 동등 조건 셀프플레이 결과

지표	결과
총 에피소드	12,000 episodes 총
학습 스텝	19,936,010 steps
Team A 승률	50.2%
Team B 승률	49.4%
무승부	0.4%
평균 FPS	~905
학습 시간	약 6.1 시간

학습 과정에서 여러 시점의 체크포인트를 저장하여 숙련도별 플레이어 모델로 활용한다 (표 10). 학습 초기 모델은 비효율적인 판단과 실수가 많아 초보 플레이어의 행동 패턴을 모사하고, 학습이 진행될수록 최적에 가까운 플레이를 보여 숙련된 플레이어를 모사한다. 최종 수렴 모델(v11)은 2 단계에서 상대팀으로 활용된다.

4.3 2 단계: 숙련도별 협동 NPC 학습

2 단계에서는 1 단계에서 저장한 체크포인트를 플레이어 역할로 활용하여 협동 NPC 를 학습한다. 플레이어가 특정 역할(예: 탱커)을 선택하면, 해당 역할에는 체크포인트 모델을 적용하고, 상대 팀(Team B)은 최종 균형 모델

그림 4. 1 단계 학습 곡선. 학습 스텝에 따른 Team A/B 승률 변화.

표 10. 체크포인트별 숙련도 분류 기준

체크포인트	v11 상대 승률	숙련도	행동 특성
1,000 ep	~15%	초보	비효율적, 실수 많음
5,000 ep	~35%	중급	기본 전략 습득
9,000 ep	~45%	고급	거의 최적
12,000 ep	50%	숙련자	완전 수렴(v11)

(v11)을 사용한다. 아군의 나머지 역할들(딜러, 힐러, 레인저, 서포터)은 해당 숙련도의 플레이어에 맞춰 협동하도록 학습시켜, 숙련도별 협동 NPC 모델을 생성한다.

이를 통해 초보 플레이어(1,000ep 체크포인트)에게는 더 적극적으로 전투를 주도하는 NPC 를, 숙련된 플레이어(v11)에게는 보조 역할에 집중하는 NPC 를 제공할 수 있다.

초기 설계에서 적 처치 보상을 최대로 설정하였을 때, NPC 가 플레이어를 무시하고 독단적으로 적에게 돌진하는 문제가 발생하였다. 이를 해결하기 위한 플레이어와의 거리 유지에 칸별 연속 보상을 부여하고, 적이 사거리내에 진입했을 때만 적극적으로 전투를 수행하도록 보상을 재설계하였다.

표 11. 2 단계 협동 학습 결과

구간	평균 보상	TeamA 승률	TeamB 승률
초기(10~100)	−139 ~ −18	30~40%	57~70%
중기(100~1000)	−18 ~ +64	40~54%	44~57%
후기(1000~3030)	+55 ~ +76	48~54%	46~52%

표 11 과 같이 약 3,030 에피소드 학습 결과, 초기 Team B 우세 상태에서 최종적으로 양 팀이 약 50:50 의 균형에 도달하였다. 평균 보상이 −139 에서 +76 으로 상승하여 NPC 가 플레이어와 효과적으로 협동하는 전략을 학습하였음을 확인하였다.

5 실제 적용 가능성 검증

학습된 NPC 모델이 실제 게임 환경에서 동작하는지 검증하기 위해 Unity 3D 데모를 구현하였다.

5.1 2D 학습에서 3D 게임으로의 적용

본 연구의 강화학습은 20x20 격자 기반 2D 환경에서 수행되었으나, 학습된 모델은 Unity 3D 게임에 직접 적용이 가능하다. 이는 관찰 벡터와 행동 공간이 게임의 시각적 표현과 독립적으로 설계되었기 때문이다. 2D 에서 3D 로의 매핑은 격자 좌표를 3D 월드 좌표로 변환하고, 이산 행동을 3D 유닛 이동과 애니메이션으로 연결하는 방식으로 구현된다. 격자 좌표(g_x, g_y)에서 Unity 월드 좌표로의 변환은 다음과 같다:

$$P_{world} = P_{origin} + (g_x * c, 0, g_y * c) \qquad (3)$$

여기서 c 는 셀 크기(기본값 1.0), P_{origin}은 그리드 원점 오프셋이다. 이를 통해 빠른 학습(단순화된 2D)과 풍부한 시각적 표현(3D)을 분리하여, 복잡한 3D 물리 시뮬레이션 없이도 전략적 행동을 학습할 수 있다.

5.2 Unity 연동 아키텍처

Unity 와 PyTorch 모델 간의 연동을 위해 이중 채널 네트워크 아키텍처를 설계하였다. 게임 상태 전송에는 UDP(Port 5005)를, 플레이어 입력 전송에는 TCP(Port 5006)를 사용한다. 이 아키텍처는 ONNX 변환 없이 PyTorch 모델을 직접 사용하며, 관전 모드와 플레이어 모드의 두 가지 운영 방식을 지원한다. Unity 측 구현은 UdpReceiver(프레임 데이터 수신), PlayerInputSender(플레이어 입력 전송), PythonManager(프로세스 생명주기 관리), GameViewer(3D 유닛 상태 갱신)등의 핵심 컴포넌트로 구성된다.

5.2.1 운영 모드

관전 모드에서는 Python 에서 양 팀 AI 가 자율적으로 전투를 수행하고 Unity 가 게임 상태를 수신하여 3D 로 시각화한다. 플레이어 모드에서는 플레이어가 A 팀의 역할 중 하나를 선택하여 직접 조작하며, 나머지 NPC 는 학습된 협동 정책에 따라 행동한다. Unity 의 Player InputSender 컴포넌트는 WASD 이동, Space 공격, Q/E/R 스킬 키 입력을 감지하여 TCP 로 Python 에 전송하며, 3D 쿼터뷰 시점에 맞게 키 매핑을 변환하여 직관적인 조작감을 제공한다. Python 은 해당 입력을 플레이어 유닛의 행동으로 반영하고, 협동 NPC 는 플레이어 위치 정보(6 차원)를 관찰 벡터에 추가로 입력받아 플레이어와의 협동 행동을 수행한다.

6 논의

본 연구가 제안하는 파이프라인의 핵심 강점은 다음과 같다. 첫째, 셀프플레이 학습 과정의 체크포인트를 활용하여 다양한 숙련도의 플레이어 모델을 자동 생성할 수 있다. 학습 초기 체크포인트는 비효율적인 행동과 실수가 많은 초보 플레이어를, 후기 체크포인트는 최적화된 전략을 구사하는 숙련 플레이어를 자연스럽게 모사한다. 이는 별도의 플레이어 데이터 수집 없이 학습 부산물만으로 숙련도별 모델을 확보할 수 있음을 의미한다. 둘째, 기존 연구[8]가 “인간을 이기는 AI”를 목표로 한 반면, 본 연구는 게임 디자인 관점에서 “유저와 시너지를 발휘하는 협동 NPC”를 목표로 한다. 숙련도별 체크포인트를 플레이어 역할로 배치하여 해당 숙련도에 최적화된 협동 NPC 를 학습함으로써, 초보 플레이어에게는 적극적으로 전투를 주도하고, 숙련된 플레이어에게는 보조 역할에 집중하는 NPC 를 제공한다. 셋째 UDP 기반 미러링으로 ONNX 변환 없이 다양한 게임 엔진에 적용 가능하다. 현재 연구의 한계점으로는 격자 기반 환경의 단순성, UDP 통신의 네트워크 지연 가능성, 실제 플레이

어 데이터와 체크포인트 행동 간의 상관관계 검증 필요성이 있다.

6.1 런타임 숙련도 판별 및 난이도 조절

런타임에서 플레이어의 숙련도를 판별하기 위해, 플레이어의 v11 상대 승률은 체크포인트 기준과 비교한다. 예를 들어 플레이어의 최근 10 게임 승률이 20%라면 1,000ep 체크포인트(15%)와 5,000ep 체크포인트(35%) 사이로 분류되어 초보-중급 경계의 협동 NPC 모델을 선택한다. 이는 객관적이고 데이터 기반의 숙련도 분류를 가능하게 한다.

게임 난이도 조절은 숙련도 적응과 별개로, 적 팀의 스탯 배율(0.7~1.2)을 조정하여 구현한다. 학습된 정책은 다양한 스탯 조건에 일반화되어, 정책 고정 상태에서 적의 HP, 공격력 등 파라미터 조절만으로 난이도 변화가 가능하다. 이는 난이도별 재학습 없이 단일 정책으로 다양한 난이도를 커버하며, 휴리스틱 기반 방식과 달리 열세에서도 전략적 행동을 유지한다. Csikszentmihalyi 의 Flow 이론 [9]과 Hunicke 의 DDA 연구 [10]에 따르면 도전과 능력의 균형이 몰입의 핵심이며, 본 파이프라인은 이를 강화학습 NPC 에 적용하여 플레이어 경험을 최적화할 수 있다.

7 결론 및 향후 연구

본 연구에서는 플레이어의 숙련도에 따라 협동 방식을 조절하는 NPC 를 강화학습으로 생성하는 2 단계 파이프라인을 제작하였다. 1 단계 동등 조건 셀프플레이에서는 12,000ep(약 20M steps) 학습을 통해 양 팀의 승률이 50.2:49.4 로 수렴하는 균형 잡힌 전투 AI(v11)를 생성하였으며, 학습 과정의 체크포인트를 저장하여 다양한 숙련도의 플레이어 모델로 활용하였다. 2 단계에서는 체크포인트를 플레이어 역할로 배치하여 해당 숙련도에 맞춰 협동하는 NPC 를 학습하고, UDP 기반 미러링 아키텍처로 ONNX 변환 없이 Unity 에서 실시간 연동이 가능함을 검증하였다. 이를 통해 초보 플레이어에게는 적극적으로 전투를 주도하는 NPC 를, 숙련된 플레이어에게는 보조 역할에 집중하는 NPC 를 제공할 수 있다.

향후 연구에서는 v11 상대 승률 기반 실시간 숙련도 판별 시스템의 정교화, 실제 플레이어 데이터와 체크포인트 행동 간의 상관관계 검증, 실제 플레이어를 대상으로 한 사용자 경험 평가, 전투 외 다른 게임 장르로의 확장을 진행할 예정이다.

참고 문헌

1. OpenAI. Dota 2 with Large Scale Deep ReinforcementLearning. arXiv preprint arXiv:1912.06680. 2019
2. Vinyals, O., Babuschkin, I., Czarnecki, W. M., et al. Grandmaster Level in StarCraft II Using Multi-Agent Reinforcement Learning. Nature, 575(7782), pp. 350 354. 2019
3. Schulman, J., Wolski, F., Dhariwal, P., Radford, A., and Klimov, O. Proximal Policy Optimization Algorithms. arXiv preprint arXiv:1707.06347. 2017.
4. Lowe, R., Wu, Y., Tamar, A., Harb, J., Abbeel, P., and Mordatch, I. Multi-Agent Actor-Critic for Mixed Cooperative-Competitive Environments. In Advances in Neural Information Processing Systems. 2017
5. Baker, B., Kanitscheider, I., Marber, T., et al. Emergent Tool Use From Multi-Agent Autocurricula. In Interna tional Conference on Learning Representations. 2020.
6. Juliani, A., Berges, V., Vckay, E., et al. Unity: A General Platform for Intelligent Agents. arXiv preprint arXiv:1809.02627. 2018.
7. Weng, J., Chen, H., Yan, D., et al. EnvPool: A Highly Parallel Reinforcement Learning Environment Execution Engine. In Advances in Neural Information Processing Systems. 2022.
8. Multi-Agent Reinforcement Learning for Creating Intel ligent Agents in Social Networks-Oriented Role Playing Games. Entertainment Computing, Volume 53. 2025.
9. Csikszentmihalyi, M. Flow: The Psychology of Optimal Experience. Harper & Row. 1990.
10. Hunicke, R. The Case for Dynamic Difficulty Adjust ment in Games. In Proceedings of the ACM SIGCHI In ternational Conference on Advances in Computer Enter tainment Technology. 2005.